School of Computer Science Engineering and Technology

Lab No. - 12 Total Marks—1

Course-B. Tech.	**Type-** Core
Course Code- CSET301	**Course Name-** Artificial Intelligence and Machine Learning
Year- 2025	**Semester-** Odd
Date-	**Batch-** 2023-2027

CO-Mapping

	CO1	CO2	CO3	CO4	CO5	CO6
Q1		√		√		

Objective

Implement and compare multiple ensemble classifiers—**Bagging**, **Random Forests**, **AdaBoost**, **Gradient Boosting**, **Voting**, and **Stacking**—on the **Raisin Dataset**. Perform **hyperparameter tuning** and present a **comparative analysis** using standard classification metrics.

Dataset: Raisin Dataset
Link: https://archive.ics.uci.edu/ml/datasets/Raisin+Dataset

1) Data Pre-processing

a) Load the dataset with **Pandas** and display the **first 5 rows**.
b) Check for **missing values** and handle if present.
c) **Encode** the target: `Class` → `Kecimen=0`, `Besni=1`.
d) **Feature screening** with **Chi-Square** (for non-negative features) or **Mutual Information**. Drop the least important feature(s) and justify briefly.
e) Split into **train (80%)** and **test (20%)** using
`train_test_split(random_state=42, stratify=Class)`.

2) Bagging Classifier

a) Train a **BaggingClassifier** with base estimator = `DecisionTreeClassifier` (defaults).

b) Evaluate on test set: **Confusion Matrix, Accuracy, Precision, Recall, F1**.
c) **Hyperparameter tuning** (Grid/Randomized Search):

- `n_estimators: [50, 100, 200]`
- `max_samples: [0.5, 0.7, 1.0]`
- `max_features: [0.5, 1.0]`
- Base tree depth (via `base_estimator__max_depth: [None, 3, 5]`)
  d) Report **best params** and metrics.

3) Random Forest Classifier

a) Train a **RandomForestClassifier** .
b) Evaluate test metrics as above.
c) Tune:

- `n_estimators: [100, 200, 300]`
- `max_depth: [None, 5, 10]`
- `min_samples_split: [2, 5, 10]`
- `max_features: ["sqrt", "log2", 0.7]`
  d) Report **best params** and metrics.
  e) Include **feature importance** plot (top 10).

4) AdaBoost Classifier

a) Train **AdaBoostClassifier** with base estimator = `DecisionTreeClassifier(max_depth=1)` (decision stump).
b) Evaluate metrics.
c) Tune:

- `n_estimators: [50, 100, 200]`
- `learning_rate: [0.01, 0.1, 1.0]`
- `base_estimator__max_depth: [1, 2]`
  d) Report **best params** and metrics.

5) Gradient Boosting Classifier

a) Train **GradientBoostingClassifier** (defaults).
b) Evaluate metrics.
c) Tune:

- `n_estimators: [100, 200]`

- `learning_rate: [0.01, 0.1, 0.2]`
- `max_depth: [2, 3, 5]`
- `subsample: [0.8, 1.0]`

d) Report **best params** and metrics.

6) Voting Classifier (Hard & Soft)

a) Build **VotingClassifier** with 3 strong base models (your tuned versions recommended):

- Example: `LogisticRegression`, `RandomForest`, `GradientBoosting`.

b) Implement both **hard** and **soft** voting (for soft, ensure estimators support `predict_proba`).
c) Evaluate metrics and compare hard vs soft voting.

7) Stacking Classifier

a) Build **StackingClassifier** with at least **3 base learners** (diverse families), e.g.:

- Level-0: `KNeighborsClassifier`, `RandomForest`, `GradientBoosting`
- Meta-learner (Level-1): `LogisticRegression`

b) Evaluate metrics; compare with other ensembles.

8) Comparative Analysis

a) Create a **summary table** (rows = models; columns = Accuracy, Precision, Recall, F1).
Models to include: **Bagging, RandomForest, AdaBoost, GradientBoosting, Voting (hard/soft), Stacking**.
b) Plot **bar charts** (grouped bars) for the four metrics.

c) **Discuss**: Which model performs best and *why* (bias-variance, effect of hyperparameters, etc.).
e) Reflect on **overfitting** indications (train vs test performance; effect of `n_estimators`, `max_depth`, `learning_rate`).